* vector<class> dinamik olarak büyüyen bir dizi.
* The Big-Five olarak geliyor constructorlar.
(Pythonda copy var. Yazdığımız class'ı operator overloading yapmalıyız.

for döngüsü varsa → next, =, >, < bu operatörler implement ediyor.

Enumerable

class A: // comparable (class bu yapıda)
:

direkt ptr üzerinden buluyor.

if(A1 = B1) // O(1), ... O(n) → Adresin gösterdiği referans üzerinden alıyor.
// = = , = = =

Bu class üzerinden eşitlik yapıyorsak C++'ta operatör overloading yapmak gerekiyor.

→ A1 = B1; // O(1) → karmaşıklığı O(1)
└ Bu işimi görmüyor.

B1.f1(); //f1... A1 modified → A'yı da etkiliyor.

A1 = B1.copy() // O(1) diyemeyiz X karmaşıklığı artıyor.
O(n)
↳ gösterdiği classtaki yapıyı veriyi söylüyor.

* copy() yapınca referansı return et ve ata.

▽ Deep copy gibi bir şey kullanırsak tüm yapıyı duplicate edeceğimiz için ek iştir. Maaliyet O(n)

karmaşıklıklar,

constructor enumerable, comparable gibi konular

! Çöpçü collection → kullanmadığı veriyi geri bırakma, önemli zaman ve karmaşıklık. karmaşıklığı büyük. Bellekten bir yer alındıysa geri bırakılmalı. Bunun için mecburen O(1) dir. O(n) dir

* Copy constructor

! O(1) → bunda referans type var.

```
class A :
        constructor A {
        (ctor)

        }
A   A1;
A   B1(A1);   // O(1)
              // O(n)
```

Burun karmaşıklığı
O(1) → constructorda çok iş yapmıyor.
O(n) → constructorda ciddi bir iş var.

(default constructor)

Order O1 = new Order(); → Bu constructoru çağırıyor. Ki, logger, datetime... bunları işler.

Artık karmaşıklık O(1) değil. O(1) olması için constructorun boş olması lazım.

! Kısaca ağaçlar classlar hep enumerable, comparable gibi yapıları implement ediyor.

B1(A1); // O(1) → Referans type aşamasında yapılıyor.
O(1) → " " 'dan daha complex bir şey yapıyor. Yapıyı dublicate ediyor.

! Maaliyette constructorun içindeki yapıya bakarak karar vericez. Mesela teker teker karakter aranası yapıyor olabilir.

1.5.6

• Constructorda referans type O(1) olmayabilir. / Syf 49

• Deconstructor, bu class bu fonksiyonda bir yerde memory allocation yapıyorsa başında olayor. Class'ı, bellekten yer alıp bırakmıyorsa deconstruction.

• Garbage collection → O(n) → Bizim yapıyı bellekten tek tek alıp siler.

Giriş ders

ADT → bu yapılar operasyonu değil yapıyı gösteriyor.

ADT → access problem ona delete daha matıklı.

↳ pointer → O(1) ... O(n)

↳ array based → erişim (access → O(1)
(silme problemi)

List → add, delete, search, update.

↳ Bu listede sadece ekleme ve çıkarma yapmak istiyorum diyip FIFO'ya göre yaparsak kuyruk olur. Burda bir add var ve delete var. Tüm yapıyı dolaşırsak O(n)

↳ LIFO'ya göre yaparsak, stack, ptr ve array olmasına göre de karmaşıklık O(1) ve O(n) arasında değişiyor.

Array-Based'te problem:

↳ size: arrayin size'ını çalışma zamanında bilemeyiz. Dynamic olarak kodda vermeli.

↳ delete problemi (silinmiyor)

↳ Dinamik yapıyı resize yaparsak, complexity problemi ortaya çıkıyor.

* people.size()

Dizilerde resize için bir complexity var. Boyutu arttığı an eklemeye izin vermiyor.

* Amortized cost → Yapı array olduğunda bunun bir size'ı var. Yeni atamaya izin vermiyor. Tekrar başka bir yerde oluşturup diziyi oraya almak gerekiyor.

* Arraylerde herhaps bir insert işlemi O(1)'dir. Fakat öyle bir yerde ekleme karmaşıklığı O(n) (n dizinin boyutu) olabilir çünkü dizinin boyutu dolarsa eklemek için başka bir yere aktarmak gerekiyor (ya da referansları aktarıyor.)

- ptr arrayi değiştirip başka bir yerde daha büyük birimlik array oluşturuyor.

arrayin boyutu n.
insert (item 1) // O(1)
insert (item 2) // O(1)
⋮
insert (item n) // dizi doldu → maaliyet O(1)
insert (item n+1) // dublicate yapıyı çöpürdü maaliyet O(n) oldu.

} insert (item n+2) maaliyeti n'dir. Dizi doldu. Taşma yapıyor.

Dizi boyutunu aşınca maaliyet O(n)

↳ ⋮

Daha önce 1 birimli bir sistem karmaşıklığı vardı. Değiştirdiğimizde 2 oldu.

$$\frac{\frac{n+1 \cdot n}{n}}{n \to \text{ortalama}} \longrightarrow \frac{2n}{n} = 2$$

→ Amortized cost: array yapılarda dizi dolduğunda maaliyetin artması ve bu maaliyetin diğerlerine dağıtılması.

* Java örneğinde resizable array diye bir yapı kullanılıyor. Resizable array // ✗ int [ ]

Dizinin boyutunu 1,1 artırıyorsak o zaman maaliyeti / amortized cost değeri n/2 yani n'dir. O(n)

↳ ⋮

$$\frac{n \cdot (n+1)}{n} = \frac{n+1}{2} \Rightarrow \frac{n}{2} \to O(n)$$

görmezden gelebiliriz.

* x2 de maaliyet O(1)

Dizi doldurtan sonra her bir yapı n olarak geliyor → O(n)

* Resize yapamıyoruz. Tekrar oluşturmak lazım.
↳ Büyüme çarpanı stratejisi dizi çok büyürse
x1.1 ile bu sorunu çözebiliyoruz.

Sınav 2 chapter → ch2 ve ch3